

ReactJS

useEffect in ReactJS:

What is useEffect?

- useEffect is a React Hook used to run some code after the component renders.
- useEffect is a React Hook used to handle “side effects”.
- useEffect is a React Hook that lets you synchronize a component with an external system.

What does Side effects mean?

Things that happen outside the normal rendering process, like:

- Fetching data from an API
- Timers (setTimeout, setInterval)
- Updating the browser (like document.title)
- Connecting to external systems (WebSocket, subscriptions)

Why do we need useEffect?

Sometimes we want to do extra work *after* the UI renders. That’s where useEffect comes in.

Basic Syntax:

```
useEffect(() => {  
    // side effect code here  
}, [dependencies]);
```

It has 2 parts:

1. Function → what to do
2. Dependency array (optional) → when to do it

While the snippet of useEffect has three value:

```
4      useEffect(() => {  
5          first  
6  
7          return () => {  
8              second  
9          }  
10     }, [third])
```

first —Side Effect logic

This code runs:

- After the component renders for the first time.
- Again whenever any dependency in the array changes.

second — Cleanup function

The function returned from `useEffect` is the cleanup function.

It runs:

- Before the effect runs again due to dependency changes.
- When the component unmounts.

third — Dependency

Values listed in the dependency array tell React when to re-run the effect.

Placeholder	Meaning
first	The effect code that runs after render
second	The cleanup code that runs before re-running the effect or on unmount
third	Dependency value(s) that determine when the effect re-runs

Now, let's understand with examples:

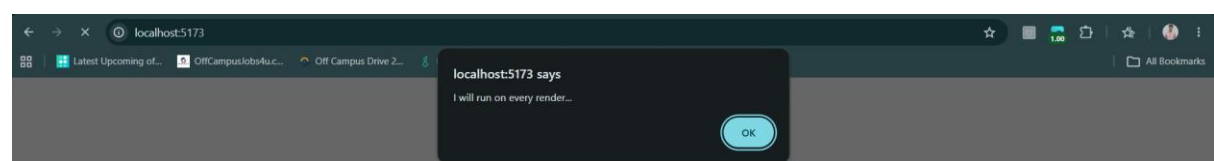
Variation 1: runs on every render.

Example:

App.js:

```
React-useeffect-project > src > App.jsx > ...
1  import { useEffect } from "react";
2
3  function App() {
4
5      useEffect(() => {
6          alert("I will run on every render...");
7      })
8
9      return (
10         <>
11             <h1>Aditya</h1>
12         </>
13     )
14 }
15
16 export default App
17
```

Output:



Since, it is in restrict mode, so it will appear again, and when clicked on “OK”, the text of <h1> will come.

Aditya

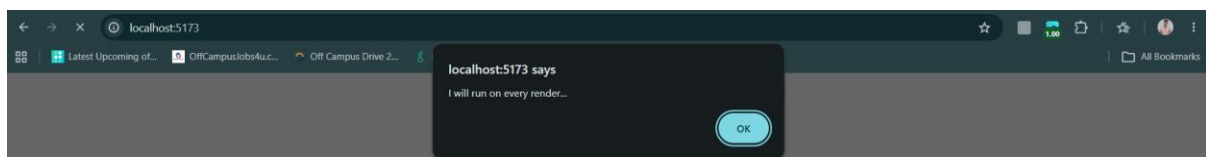
This code creates a React component called App. Inside it, useEffect() is used without a dependency array, which means the code inside it runs after every render of the component. In this case, it shows an alert saying "I will run on every render..." each time the component renders. The component also displays the heading "Aditya" on the page. Since the effect runs on every render, the alert will keep appearing whenever the component re-renders.

Another example of “runs on every render”.

App.js:

```
React-useeffect-project > src > App.jsx > App
1  import { useEffect, useState } from "react";
2
3  function App() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      alert("I will run on every render...");
8    })
9
10   function handleClick(){
11     setCount(count+1);
12   }
13   return (
14     <>
15       <h1>Aditya</h1>
16       <button onClick={handleClick}>Click Me !</button>
17       <br/>
18       <h2>Count: {count}</h2>
19     </>
20   )
21 }
22 export default App
```

Output: this comes on first render. When the screen loaded for the first time.



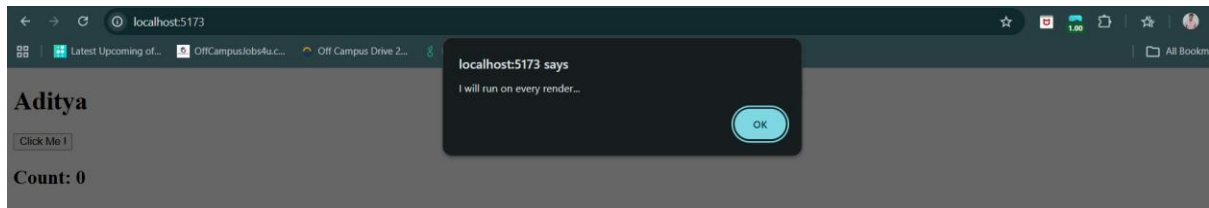
When clicked OK:

Aditya

Click Me !

Count: 0

When clicked on the “Click Me!” button: again it aimed to re-render the count from 0 to 1, but since `useEffect` is used, it will run before the count to increase from 0 to 1.



Once, we click “OK” again: count get increased from 0 to 1.



This component displays a name, a button, and a counter. The count state starts at 0. When you click the "Click Me!" button, the `handleClick` function increases the count by 1 using `setCount()`. Since updating state causes the component to re-render, the `useEffect()` hook runs again. Because no dependency array is provided, the effect runs after every render, so the alert "I will run on every render..." appears when the component first loads and again every time the button is clicked and the count changes.

Now, other variations..

Variation 2: that runs on only the first render.

Basically, the main logic snippet be this:

```
useEffect(() => {  
  alert("I will run on only the first render...");  
}, [])
```

When the page loads, the alert appears once. Clicking a button or updating state later will re-render the component, but the `useEffect` will not run again because there are no dependencies to watch for changes.

Let's see this using an example:

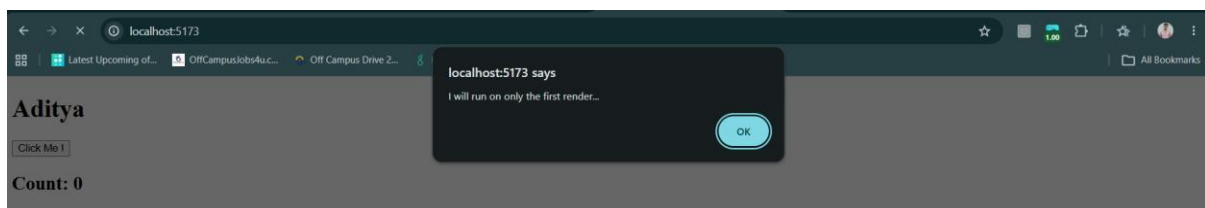
App.js:

```

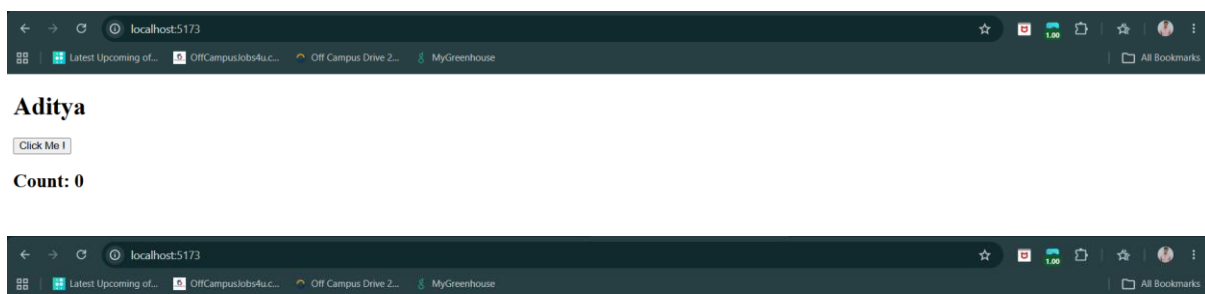
React-useeffect-project > src > App.jsx > App
1  import { useEffect, useState } from "react";
2
3  function App() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      alert("I will run on only the first render...");
8    }, [])
9
10   function handleClick(){
11     setCount(count+1);
12   }
13   return (
14     <>
15       <h1>Aditya</h1>
16       <button onClick={handleClick}>Click Me !</button>
17       <br/>
18       <h2>Count: {count}</h2>
19     </>
20   )
21 }
22 export default App
23

```

Output: when it loaded for first time.



When clicked OK, and clicked on the "Click Me!" button: no alert comes again, simply the count increases from 0 to 1, and so on.



This component uses `useState` to keep track of a counter that starts at 0. The `useEffect` hook has an empty dependency array (`[]`), so it runs only once, right after the component is rendered for the first time. As a result, the alert "I will run on only the first render..." appears only when the page initially loads. When the "Click Me!" button is clicked, the count state increases and the component re-renders to show the updated count, but the `useEffect` does not run again because its dependency array is empty.

Variation 3: there is something inside the dependencies.

Main logic snippet;

```
useEffect(() => {  
  alert("I will run everytime when count is updated");  
},[count])
```

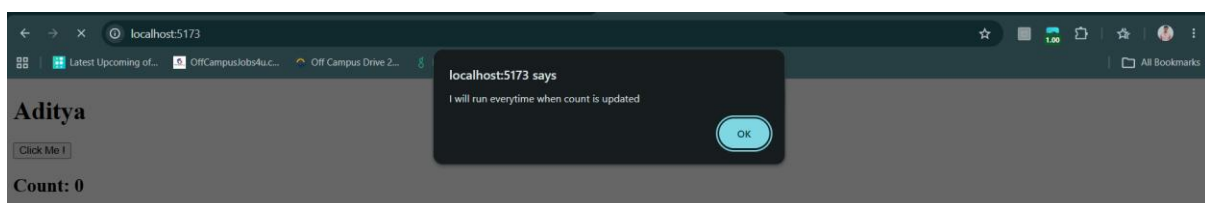
React watches count, and whenever count changes, it executes the code inside useEffect.

Example:

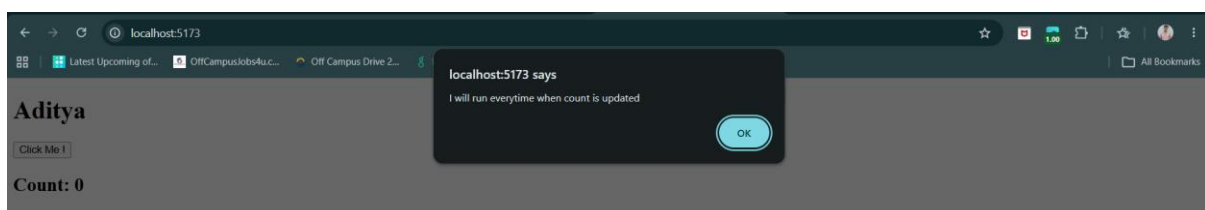
App.jsx:

```
React-useeffect-project > src > App.jsx > App  
1  import { useEffect, useState } from "react";  
2  
3  function App() {  
4    const [count, setCount] = useState(0);  
5  
6    useEffect(() => {  
7      alert("I will run everytime when count is updated");  
8    }, [count])  
9  
10   function handleClick(){  
11     setCount(count+1);  
12   }  
13   return (  
14     <>  
15       <h1>Aditya</h1>  
16       <button onClick={handleClick}>Click Me !</button>  
17       <br/>  
18       <h2>Count: {count}</h2>  
19     </>  
20   )  
21 }  
22 export default App
```

Output: when screen loaded, the count was assigned '0' and thus the logic of useEffect was executed as well.



When clicked on "OK" and then clicked on the "Click Me!" button: the alert box will come again.



When clicked "OK": count will get updated from 0 to 1. And for every update in the 'count' there will be Alert box coming.



Aditya

Click Me!

Count: 1

This component displays a counter that starts at 0. The `useEffect` hook has `count` in its dependency array (`[count]`), so React runs the effect after the component first renders and then every time the `count` value changes. When you click the "Click Me!" button, the `handleClick` function increases the `count` by 1 using `setCount(count + 1)`. This updates the state, causes a re-render, and since `count` changed, the `useEffect` runs again and shows the alert "I will run everytime when count is updated". As a result, you'll see the alert when the page first loads and after each button click.

Variation 4: multiple dependencies.

```
useEffect(() => {  
  alert("I will run everytime when count/total is updated");  
},[count,total])
```

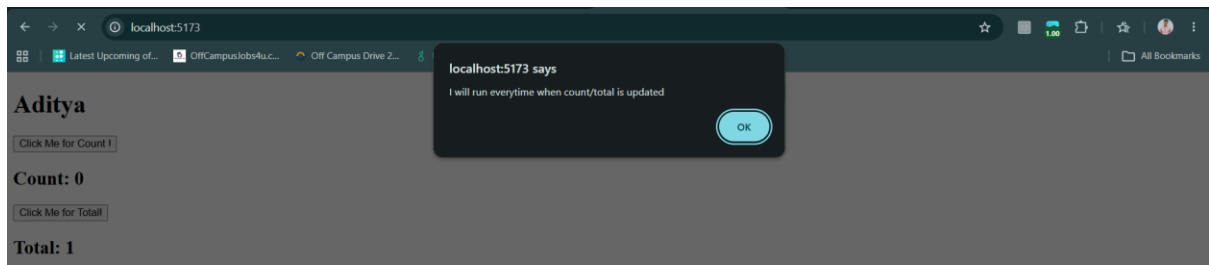
React watches both `count` and `total`. Whenever either one is updated, the code inside `useEffect` executes.

Example:

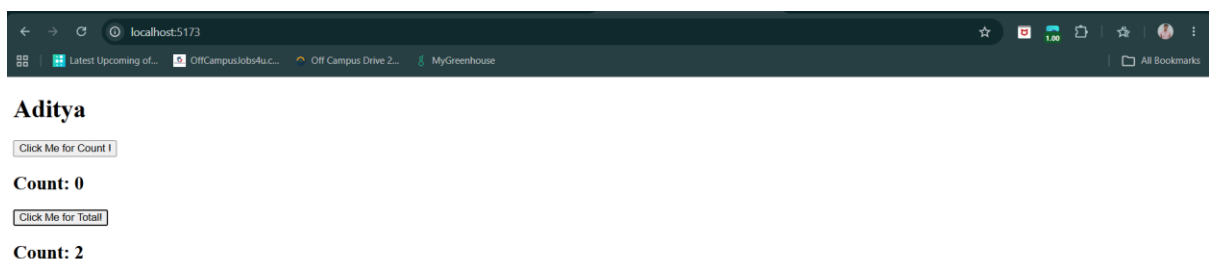
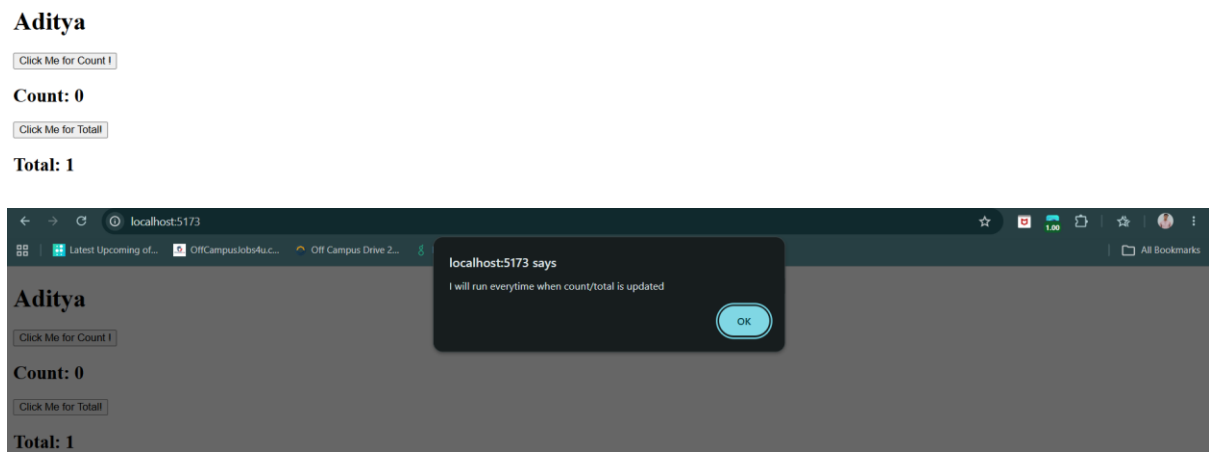
App.jsx:

```
React-useeffect-project > src > App.jsx > App  
1  import { useEffect, useState } from "react";  
2  
3  function App() {  
4    const [count, setCount] = useState(0);  
5    const [total, setTotal] = useState(1);  
6  
7    useEffect(() => {  
8      alert("I will run everytime when count/total is updated");  
9    }, [count, total])  
10  
11    function handleCount() {  
12      setCount(count + 1);  
13    }  
14    function handleTotal() {  
15      setTotal(total + 1);  
16    }  
17    return (  
18      <>  
19        <h1>Aditya</h1>  
20        <button onClick={handleCount}>Click Me for Count !</button>  
21        <br/>  
22        <h2>Count: {count}</h2>  
23        <button onClick={handleTotal}>Click Me for Total!</button>  
24        <br/>  
25        <h2>Count: {total}</h2>  
26      </>  
27    )  
28  }  
29  export default App
```

Output:



When clicked on “OK” and then clicked on either of the buttons: again the alert will come.



This component uses two state variables: count, which starts at 0, and total, which starts at 1. The useEffect hook has [count, total] as its dependency array, so it runs once when the component first renders and then every time either count or total changes. Clicking "Click Me for Count!" increases the count value, while clicking "Click Me for Total!" increases the total value. Whenever either button is clicked, the component re-renders and the alert "I will run everytime when count/total is updated" appears because one of the dependencies has been updated. The current values of count and total are displayed on the screen and update automatically after each click.

Variation 5: With cleanup function

Main code snippet:

```
useEffect(() => {  
  alert("Count is updated");  
  return ()=>{  
    alert("count is unmounted from UI")  
  }  
},[count])
```

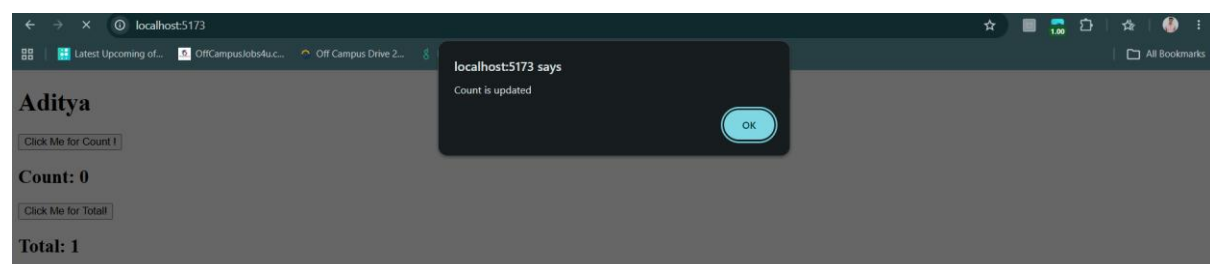
This `useEffect` runs when the component first loads and whenever `count` changes. When `count` is updated, it first runs the cleanup function from the previous effect, showing the alert "count is unmounted from UI", and then runs the new effect showing "Count is updated". The cleanup is not actually about unmounting the whole component here—it runs before the effect runs again due to dependency changes. So in simple terms: whenever `count` changes, React first clears the previous effect, then runs the new one.

Example:

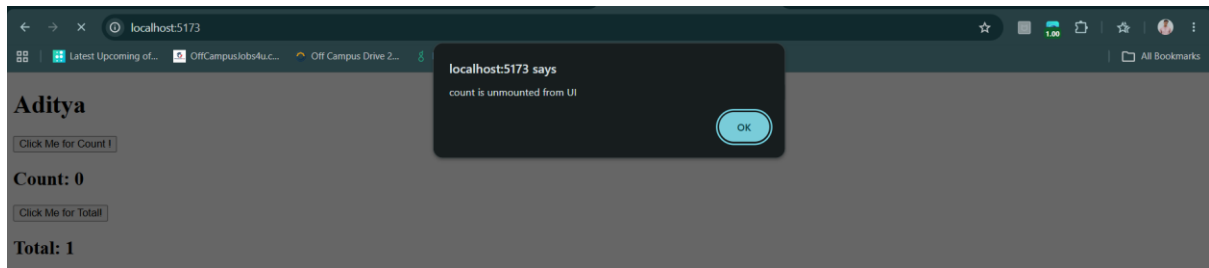
App.jsx:

```
React-useeffect-project > src > App.jsx > App  
1  import { useEffect, useState } from "react";  
2  
3  function App() {  
4    const [count, setCount] = useState(0);  
5    const [total, setTotal] = useState(1);  
6    useEffect(() => {  
7      alert("Count is updated");  
8      return ()=>{  
9        alert("count is unmounted from UI")  
10     }  
11    },[count])  
12  
13    function handleCount(){  
14      setCount(count+1);  
15    }  
16    function handleTotal(){  
17      setTotal(total+1);  
18    }  
19    return (  
20      <>  
21        <h1>Aditya</h1>  
22        <button onClick={handleCount}>Click Me for Count !</button>  
23        <br/>  
24        <h2>Count: {count}</h2>  
25        <button onClick={handleTotal}>Click Me for Total!</button>  
26        <br/>  
27        <h2>Total: {total}</h2>  
28      </>  
29    )  
30  }  
31  export default App
```

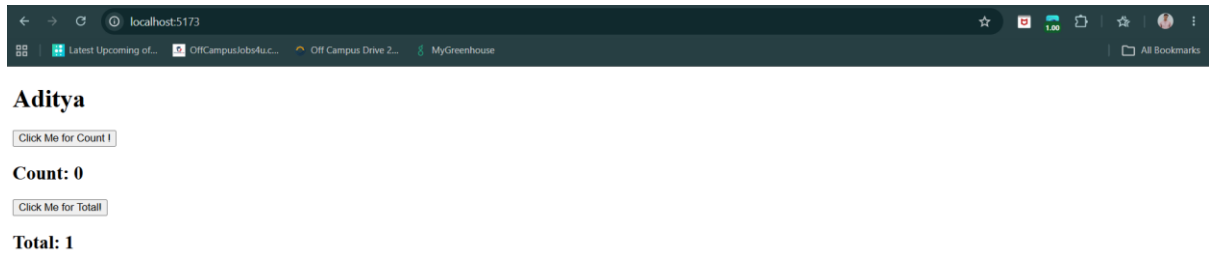
Output: when the screen loaded.



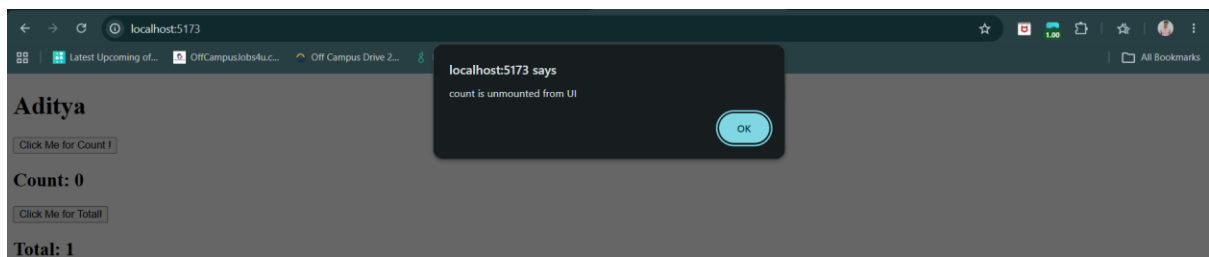
When clicked on “OK”, the 2nd alert box came:



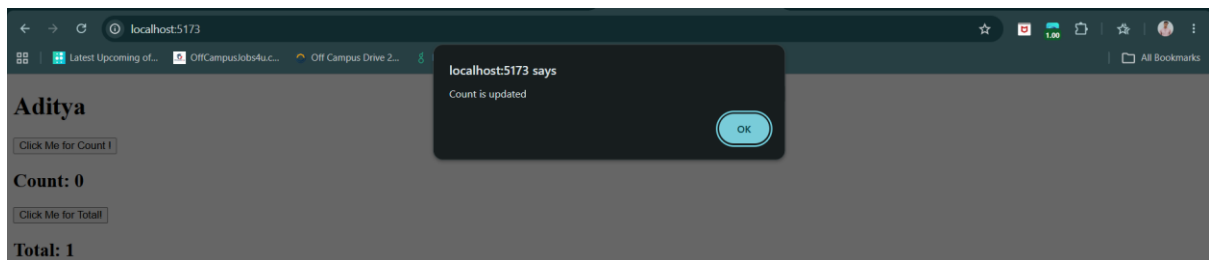
When clicked on “OK” again:



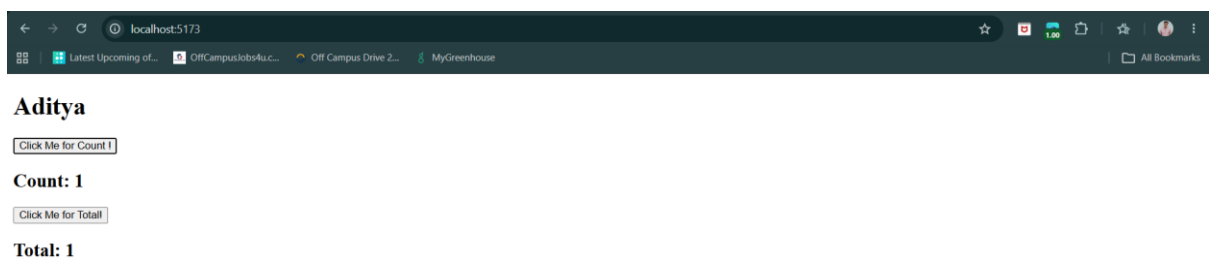
When clicked on to update the count: the “count is unmounted from UI” is displayed first, this is because the previous value of count is unmounted now before updating.



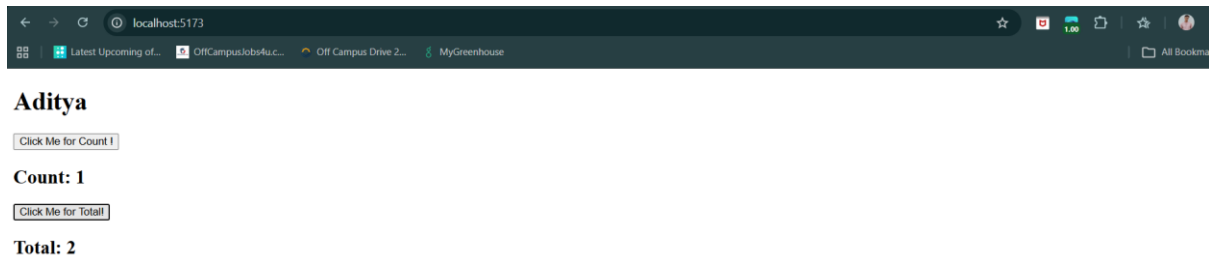
Then the count is updated alert box will come:



Then the count is updated from 0 to 1:



Also, when clicked to increase the “Total”: no alert box will come.



Use Effect Hook for Life Cycle Methods in React:

What are the phases of life cycle in react?

In React, a component lifecycle means the different stages a component goes through from the time it is created until it is removed from the screen.

Think of it like a person's life:

1. Birth → Component is created.
2. Growth → Component updates when data changes.
3. Death → Component is removed.

Phases of React Component Lifecycle:

1. Mounting Phase (Component is Created): This phase happens when the component is added to the screen for the first time.

2. Updating Phase (Component Changes): This phase happens when:

- State changes (setState)
- Props change
- Force update occurs

3. Unmounting Phase (Component is Removed): This phase happens when the component is removed from the screen.

useEffect Syntax	Lifecycle Phase
useEffect(() => {}, [])	Mounting
useEffect(() => {}, [state/props])	Updating
useEffect(() => { return () => {} }, [])	Unmounting
useEffect(() => {})	Every render (Mount + Update)

useContext in ReactJS:

Let's first understand the concept of Prop Drilling: (even before it, revisiting Props)

What is Prop?

Props (short for *properties*) are used to pass data from a parent component to a child component.

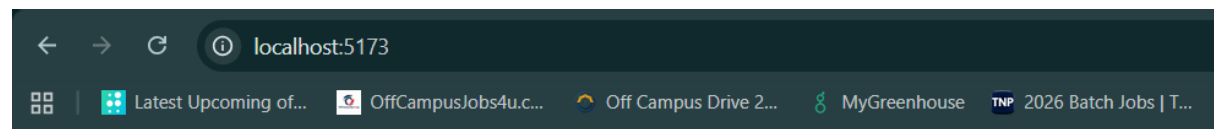
Example:

```
src > App.jsx > App
1  function Child(props) {
2    |   return <h1>Hello {props.name}</h1>;
3    | }
4
5  function App() {
6    |   return <Child name="Aditya" />;
7    | }
```

Here:

- App is the parent.
- Child is the child.
- name="Aditya" is passed as a prop.

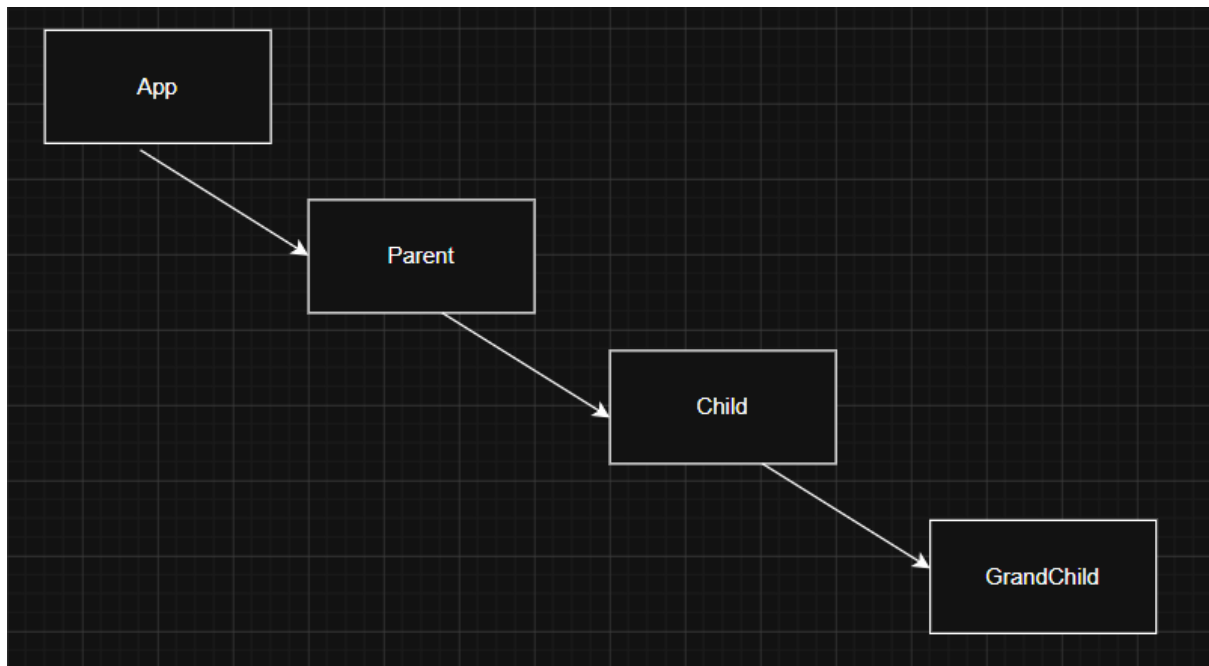
Output:



Hello Aditya

What Problem Does Prop Drilling Solve?

Imagine you have this component structure:



Suppose a value is stored in App, but GrandChild needs it. Since React data normally flows parent → child, the value must travel through every intermediate component. This process is called Prop Drilling.

So, what is prop drilling?

Prop Drilling is the process of passing data from a parent component to a deeply nested child component through multiple levels of components using props. Even if intermediate components don't need the data, they still have to pass it down.

Why prop drilling can become a problem?

1. Lots of Repeated code
2. Harder to maintain
3. Intermediate Components Become Unnecessary Messengers

Is Prop Drilling a React Hook?

No. Prop Drilling is not a hook. It is simply a pattern of passing props through multiple levels.

Difference Between Prop Drilling and Hooks:

Prop Drilling	Hooks
Not a hook	Special React functions
Used to pass data through components	Used to add React features
Doesn't manage state itself	Can manage state and side effects
Uses props	Uses hooks like useState, useEffect, useContext
Can become messy in deep trees	Some hooks help avoid prop drilling

Example: a basic example of prop drilling.

App.jsx:

```
src > App.jsx > ...
1  import Parent from "../myComponent/Parent";
2
3  function App() {
4    const theme = "Dark Mode";
5
6    return (
7      <>
8        <h1>App Component</h1>
9        <Parent theme={theme} />
10     </>
11   );
12 }
13
14 export default App;
```

Parent.jsx:

```
src > myComponent > Parent.jsx > ...
1  import Child from "../Child";
2
3  function Parent({ theme }) {
4    return (
5      <>
6        <h2>Parent Component</h2>
7        <Child theme={theme} />
8      </>
9    );
10 }
11
12 export default Parent;
```

Child.jsx:

```
src > myComponent > Child.jsx > [⌕] default
1  import GrandChild from "../GrandChild";
2
3  function Child({ theme }) {
4    return (
5      <>
6        <h3>Child Component</h3>
7        <GrandChild theme={theme} />
8      </>
9    );
10 }
11
12 export default Child;
```

GrandChild.jsx:

```
src > myComponent > GrandChild.jsx > default
1  function GrandChild({ theme }) {
2    return (
3      <>
4        <h4>GrandChild Component</h4>
5        <p>Theme: {theme}</p>
6      </>
7    );
8  }
9
10 export default GrandChild;
```

Output:

App Component

Parent Component

Child Component

GrandChild Component

Theme: Dark Mode

Step-by-step

1. App creates:

```
const theme = "Dark Mode";
```

2. App passes it to Parent:

```
<Parent theme={theme} />
```

3. Parent receives it:

```
function Parent({ theme })
```

and passes it to Child:

```
<Child theme={theme} />
```

4. Child receives it:

```
function Child({ theme })
```

and passes it to GrandChild:

```
<GrandChild theme={theme} />
```

5. GrandChild finally uses it:

`<p>Theme: {theme}</p>`

So, basically prop drilling is an issue, which can be overcome using the useContext hook or also called useContext API.

What is useContext?

useContext is a React Hook that lets a component access data from a Context without passing that data through props at every level.

useContext has three basic steps:

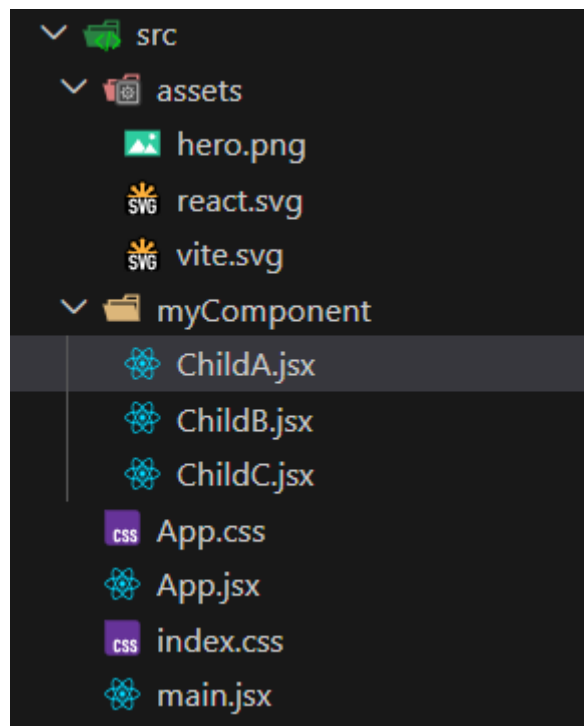
1. Create Context
2. Provide
3. Consume

So, for earlier problem:

Instead of passing props through every component, GrandChild can directly ask React: *"Give me the user stored in Context."* No prop drilling needed.

Now, let's understand the steps of useContext step by step:

First see the folder structure for this:



Step1: Create Context

```
src > App.jsx > App
1  import { createContext } from "react"
2  const UserContext = createContext();
```

This creates a "global container".

Step2: Wrap all the child inside a provider. Basically, we are wrapping every child which we want to be consumer.

```
<UserContext.Provider>
|   <ChildA/>
| </UserContext.Provider>
```

So, now App.jsx, looks like this:

```
src > App.jsx > default
1  import { createContext } from "react"
2  import ChildA from "../myComponent/ChildA";
3  const UserContext = createContext();
4
5  function App() {
6    return (
7      <>
8        <UserContext.Provider>
9          <ChildA/>
10        </UserContext.Provider>
11      </>
12    )
13  }
14
15  export default App
```

Step3: Pass Value

For this first create the value:

```
const [user, setUser] = useState({name: "Love"});
```

Now, passing the value:

```
<>
<UserContext.Provider value={user}>
|   <ChildA/>
| </UserContext.Provider>
</>
```

So, now the App.jsx: notice that we are exporting the "UserContext" as well.

```
src > App.jsx > ...
1  import { createContext, useState } from "react"
2  import ChildA from "../myComponent/ChildA";
3  const UserContext = createContext();
4
5  function App() {
6    const [user, setUser] = useState({name: "Love"});
7    return (
8      <>
9        <UserContext.Provider value={user}>
10          <ChildA/>
11        </UserContext.Provider>
12      </>
13    )
14  }
15
16  export default App
17  export {UserContext}
```

This code creates a shared user object ({name: "Love"}) and makes it available to ChildA and all its child components through React Context.

Step4: Consume value.

For this, the code of the child be given below:

ChildA.jsx:

```
src > myComponent > ChildA.jsx > ...
1  import React from 'react'
2  import ChildB from './ChildB'
3
4  function ChildA() {
5    return (
6      <div>
7        <ChildB/>
8      </div>
9    )
10 }
11
12 export default ChildA
```

ChildB.jsx:

```
src > myComponent > ChildB.jsx > ChildB
1  import React from 'react'
2  import ChildC from './ChildC'
3
4  function ChildB() {
5    return (
6      <div>
7        <ChildC/>
8      </div>
9    )
10 }
11
12 export default ChildB
```

ChildC.jsx: here we are using the value.

```
src > myComponent > ChildC.jsx > ChildC
1  import React, { useContext } from 'react'
2  import { UserContext } from '../App'
3
4  function ChildC() {
5    const user = useContext(UserContext);
6    return (
7      <div>
8        <h1>{user.name}</h1>
9      </div>
10 )
11 }
12
13 export default ChildC
```

Output:

Love

Now, let's understand the useContext with another example:

App.jsx:

```
src > App.jsx > ...
1  import { createContext, useState } from "react"
2  import ChildA from "../myComponent/ChildA";
3  const ThemeContext = createContext();
4  import './App.css'
5
6  function App() {
7    const [theme, setTheme] = useState('light');
8    return (
9      <>
10     <ThemeContext.Provider value={{theme, setTheme}}>
11       <div id='container' style={{backgroundColor: theme==='light'?"beige":"black"}}>
12         <ChildA/>
13       </div>
14     </ThemeContext.Provider>
15   </>
16 )
17 }
18 export default App
19 export {ThemeContext}
```

ChildA.jsx:

```
src > myComponent > ChildA.jsx > ChildA
1  import React from 'react'
2  import ChildB from './ChildB'
3
4  function ChildA() {
5    return (
6      <div>
7        <ChildB/>
8      </div>
9    )
10 }
11
12 export default ChildA
```

ChildB.jsx:

```
src > myComponent > ChildB.jsx > ...
1  import React from 'react'
2  import ChildC from './ChildC'
3
4  function ChildB() {
5    return (
6      <div>
7        <ChildC/>
8      </div>
9    )
10 }
11
12 export default ChildB
```

ChildC.jsx:

```
src > myComponent > ChildC.jsx > default
1  import React, { useContext } from 'react'
2  import { ThemeContext } from '../App'
3
4  function ChildC() {
5    const {theme, setTheme} = useContext(ThemeContext);
6    function handleClick(){
7      if(theme==='light')
8        setTheme('dark')
9      else
10       setTheme('light')
11    }
12    return (
13      <div>
14        <button onClick={handleClick}>
15          Change theme
16        </button>
17      </div>
18    )
19  }
20
21  export default ChildC
22
```

Output:



Output: if clicked.



How useContext is different from other hooks?

Hook	Purpose
useState	Store component state
useEffect	Perform side effects
useRef	Store mutable values without re-render
useMemo	Memoize expensive calculations
useCallback	Memoize functions
useContext	Read shared context data

Styling in React JS:

There are 5 common ways to style React applications:

1. Inline Style
2. External CSS
3. CSS Modules
4. Styled Components
5. External CSS Libraries/Frameworks (Bootstrap, Tailwind CSS, Material UI)